File formats

This document explains the file formats in src/formats (OFS::Formats) byte by byte, the way the code reads and writes them, along with the odd things found in real files.

We started from Elecbyte's M.U.G.E.N documentation and from how IKEMEN GO (MIT) reads the files. Then every detail below was checked against real files:

- the sample data that comes with Fighter Factory Studio, including its Kung Fu Man (an SFF 2.01 file with 281 sprites and 7 palettes, and an SND with 12 sounds);
- a M.U.G.E.N 1.0 folder with lots of characters, stages, fonts and screenpacks made by other people.

That makes 195 SFF files (75 v1, 114 v2.0, 6 v2.01) with 168,338 sprites, 96 SND files with 17,096 sounds and 70 FNT files. The numbers further down come from these files. Elecbyte's command line tools sff2png.exe (v0.1), sprmake2.exe (2.00beta) and sndmaker.exe (1.02) were used to check our results (see "Checks against Elecbyte's tools" at the end).

None of these files are in the repository. The tests that use them read their folders from OFS_CORPUS_KFM and OFS_CORPUS_MUGEN, and are skipped when those aren't set.

Numbers that take more than one byte are little-endian, and offsets count from the start of the file, unless it says otherwise.

SFF sprite files

sff/SffDocument.h. One class (SffDocument) holds every version. SffDocument::version decides which layout write() uses, and convertTo() changes it.

Version bytes

Bytes 12-15 hold the version as build, minor 2, minor 1, major. So version 2.0.1.0 is stored as 00 01 00 02. These are the ones found in real files:

Stored bytes	Version	Meaning
00 01 00 01	1.0.1.0	SFF v1 (WinMUGEN)
00 00 00 02	2.0.0.0	SFF v2 (M.U.G.E.N 1.0)
00 01 00 02	2.0.1.0	SFF v2.01 (M.U.G.E.N 1.1, adds PNG sprites)

The class keeps the four bytes as they were read (versionBytes) next to the version number.

SFF v1

The header is 512 bytes:

Offset	Size	Field
0	12	"ElecbyteSpr\0"
12	4	version bytes 00 01 00 01
16	4	number of groups

Offset	Size	Field
20	4	number of images
24	4	offset of the first subfile (512)
28	4	size of a subfile header (32)
32	1	palette type: 1 = shared, 0 = each sprite has its own
33	3	unused
36	476	unused, often holds a comment

The images are a chain of subfiles. Each one has a 32-byte header and then the image data:

Offset	Size	Field
0	4	offset of the next subfile
4	4	length of the image data, 0 for a linked sprite
8	2	axis x (signed)
10	2	axis y (signed)
12	2	group
14	2	image number
16	2	for a linked sprite: the index of the sprite it shares
18	1	1 = use the palette of the sprite before ("same palette")
19	13	unused

The image data is an 8-bit PCX file (see PCX below). A sprite with "same palette" set can leave out the PCX palette. It is then drawn with the palette of the last sprite before it that has its own pixels (linked sprites don't count). A linked sprite shows the pixels and the palette of the sprite it points to. Its PCX data is empty, so the next subfile starts right after its header.

In SffDocument, every sprite with its own palette adds one SffPalette (named after the sprite's group and number), and each sprite's paletteIndex points to the palette it's drawn with. That way v1 and v2 files are handled the same.

Odd things found in real files:

- **Wrong lengths.** In 153 subfiles of 8 files (Elecbyte's fight.sff, fightfx.sff, system.sff and Kung Fu Man's intro.sff, plus the intro and ending of one other character) the length is exactly 768 bytes too big. It still counts the palette that was taken out because the sprite uses the palette before it. In the last subfile this even points past the end of the file. So the reader finds the end of the data from the next subfile's offset, and only uses the length when that offset can't be used.
- In the last subfile, the next subfile offset is the file size (never 0).
- The number of groups can't be trusted (one stage stores 0 for 4 groups).
- Two files store 512 as the subfile header size, but the headers are 32 bytes.
- 475 sprites with "same palette" set still have a full palette, which isn't used.
- 1,257 PCX images pad their lines to a multiple of 4 bytes (it's usually 2).
- Lots of files have a comment in the header (60 files) or bytes other than 0 in the unused part of subfile headers (1,755 subfiles).

Writing: subfiles are written in order, and each next offset points right after the data. A sprite whose palette is the same as the one before (same RGB) gets "same palette" and a PCX without a palette. Sprites that came from a file and weren't changed are written with their original PCX bytes, length, link, "same palette" byte and unused bytes. If only the palette of such a sprite changed, the 768 palette bytes at the end of its PCX are replaced (or added) and the image isn't encoded again. The header is written from the 512 bytes that were read, with the counts and offsets updated. The original number of groups and subfile header size stay as they were as long as the groups don't change.

SFF v2 (2.0 and 2.01)

The header is 512 bytes:

Offset	Size	Field
0	12	"ElecbyteSpr\0"
12	4	version bytes
16	8	reserved (0)
24	4	"compatible" version bytes, the same as the version in every file we've seen
28	8	reserved (0)
36	4	offset of the sprite node table
40	4	number of sprites
44	4	offset of the palette node table
48	4	number of palettes
52	4	offset of the literal data block
56	4	length of the literal data block
60	4	offset of the translated data block
64	4	length of the translated data block
68	8	reserved (0)
76	436	unused, some editors put a comment here

Palette node (16 bytes):

Offset	Size	Field
0	2	group
2	2	number
4	2	number of colors (32 or 256 in practice)
6	2	for a linked palette: the index of the palette it shares
8	4	offset of the colors inside the literal block
12	4	length in bytes (colors x 4), 0 for a linked palette

Colors are stored as R, G, B and a fourth byte. Programs don't agree on that byte. Elecbyte's sprmake2 2.00 writes 0. Most palettes we've seen (made with other editors, Fighter Factory's Kung Fu Man too) have 0 for index 0 and 255 for the rest, and a few have other values. SffDocument keeps it in Rgba::a so it can be written back, and ignores it everywhere else (index 0 is see-through, the rest are solid).

Sprite node (28 bytes):

Offset	Size	Field
0	2	group
2	2	number
4	2	width
6	2	height
8	2	axis x (signed)
10	2	axis y (signed)
12	2	for a linked sprite: the index of the sprite it shares
14	1	format, see below
15	1	color depth: 5 or 8 (indexed), 24 or 32 (true color)
16	4	offset of the data inside its block
20	4	length of the data, 0 for a linked sprite
24	2	palette index
26	2	flags: bit 0 set = the data is in the translated block

Format	Name	Data
0	raw	width x height palette indices, nothing in front (raw 24 or 32-bit data is refused: we've never seen it and SprMaker2 can't make it, so its byte order is unknown)
1	(invalid)	never used for data, some programs store it in linked nodes
2	RLE8	4-byte unpacked size, then RLE8 data
3	RLE5	4-byte unpacked size, then RLE5 data
4	LZ5	4-byte unpacked size, then LZ5 data
10	PNG8	4-byte size (width x height), then an 8-bit palette PNG
11	PNG24	4-byte size (width x height x 3), then an RGB PNG
12	PNG32	4-byte size (width x height x 4), then an RGBA PNG

Data in the literal block is used as it is. Data in the translated block is unpacked by the engine when it loads the file (SprMaker2's sprite.decompressonload). Only packed formats show up there in real files (we've seen RLE8 and LZ5). sff2png won't open a file with a raw sprite in the translated block, so the writer never puts new raw data there.

For PNG8 sprites only the PNG's pixel indices are used. The sprite is drawn with the SFF palette its node names, and the PNG's own PLTE is ignored. The writer still puts the SFF palette in the PLTE, so the PNG looks right in other programs.

Every file we've seen (and every file we write) is laid out like this: the palette node table at 512, the sprite node table right after it, then the literal block with all the palette data first (in palette order) and then the sprite data in node order, and last the translated block, which ends at the end of the file.

Links:

- A linked palette or sprite has length 0 and the index of an earlier node. A sprite can link to a sprite that is itself linked.
- The data offset of a linked node isn't used, and files fill it in one of two ways and stick to it. 119 of the 120 v2 files store where the next data would go in the node's block (a "running offset"). Fighter Factory's Kung Fu Man stores the offset of the target's data. SffDocument remembers which one a file uses (linkOffsets) and writes links the same way.
- The format byte of a linked sprite is either 1 or a copy of the target's format. The color depth is usually the target's (9 sprites have a different one).
- A linked sprite has its own palette index, and in 698 sprites it's different from the target's. IKEMEN GO draws the link with its own palette, and so does decodeSprite(). Elecbyte's sff2png does too, except when that palette's size is different from the target palette's (for example a 5-bit sprite shown through a link whose palette has 256 colors). Then it uses the target's palette. Files written by our tools keep link palettes at the size the sprite needs.

Odd things found in real files:

- A font file (num72.sff) says version 2.0.0.0 but has PNG32 sprites. Files like that are treated as v2.01 (version), and the stored version bytes are kept.
- 97 files have text in the unused part of the header ("Made in ... Powered by Fighter Factory").
- 13 RLE8 sprites in one character (SSJ4_GogetaZ2i) end long before all the pixels are there (the data ends in zeros). They're read in lenient mode, with the missing pixels set to 0. Every other sprite reads fine in strict mode.

Writing: palettes and sprites are laid out as above. A sprite that came from a file and wasn't changed keeps its stored bytes, format and depth. A palette keeps its colors, fourth byte included, and links keep their indices and stored fields. With the stored header, this gives back every file we tested byte for byte. New or edited sprites are encoded the way SffCompressionPolicy says: by default RLE8 for 8-bit, LZ5 for sprites that only use indices 0-31 and PNG32 for true color, or else whichever encoding comes out smallest. SFF 2.0 can't store PNG, so those sprites are encoded again, and true color sprites are refused. linkDuplicateSprites and linkDuplicatePalettes turn sprites with the same pixels and palettes with the same colors into links.

Changing the version

convertTo() changes the version:

- v1 to v2: palettes that are the same are merged (v1 stores one for each sprite). The palette of sprite 0,0 (or of the first sprite) becomes palette 1,1, so M.U.G.E.N uses the character's ACT files for it. The PCX sprites are encoded again when saving.
- v2 to v1: a link whose palette is different from its target's becomes a copy, because v1 links always share the palette. True color sprites can't be changed (use adaptToPalette() first).
- 2.01 to 2.0: PNG8 sprites are encoded again, and true color is refused.

Sprite codecs

codecs/. Each decoder takes the packed data (without the SFF size in front) and the number of pixels to make. In DecodeMode::Strict, data that runs out, runs that go past the end of the image

and copies from before its start are errors. DecodeMode::Lenient works like the engines: missing pixels stay 0 and runs that are too long are cut. Extra bytes after the last pixel are ignored.

RLE8

A byte whose two top bits are 01 (0x40-0x7F) starts a run. The low six bits are the run length (0-63) and the next byte is the value. Any other byte is one pixel. So the values 0x40-0x7F always take two bytes, even on their own. The encoder writes runs of up to 63, and single pixels as one byte when it can.

RLE5

For indices 0-31. A packet is:

Bytes	Meaning
1	run length R of the first color
1	bit 7: a first color follows; bits 0-6: number N of extra runs (0-127)
0 or 1	the first color when bit 7 is set, otherwise the first color is 0
N	extra runs, one byte each: bits 5-7 = length - 1, bits 0-4 = color

The first color is repeated R + 1 times (1-256), and each extra run its length (1-8). None of our test files use RLE5, so the decoder was checked against sprmake2 and sff2png.

sprmake2 bug: SprMaker2 2.00beta sometimes writes a packet with 128 extra runs. That number doesn't fit in seven bits and spills into the color flag, so the byte reads as "first color follows, no extra runs" and the rest of the image comes out different from what was meant. Elecbyte's own sff2png reads these packets exactly like we do, so the sprites look the same as in M.U.G.E.N's tools. Our encoder never writes more than 127 extra runs.

LZ5

For indices 0-31. Packets come in groups of up to eight, and each group starts with a control byte. Bit k (lowest first) says whether packet k is a run (0) or a copy (1).

Packet	Bytes	Encoding
short run	1	bits 5-7 = length (1-7), bits 0-4 = color
long run	2	the first byte has bits 5-7 clear and holds the color, the second is length - 8 (8-263)
short copy	1 or 2	bits 0-5 = length - 1 (2-64, never 0), bits 6-7 see below, then distance - 1
long copy	3	first byte: bits 0-5 clear, bits 6-7 = bits 8-9 of distance - 1; then bits 0-7 of distance - 1; then length - 3 (3-258)

A copy repeats the output from distance pixels back (1-256 for short copies, 1-1024 for long ones), and the part it copies from can overlap the part it writes. The top two bits of each short copy are collected, two at a time and highest first, into a spare byte. The first three short copies have their own distance byte. The fourth has none and uses the collected byte as its distance - 1. Collecting starts again after every fourth short copy, and control bytes don't reset it.

The data ends after the last packet. No control byte follows, even when the last packet fills a group of eight (1,096 of the 6,891 LZ5 sprites we've seen end like that, and none has extra bytes).

The encoder finds matches with hash chains over pixel pairs (window 1024, chain depth 48), and picks a run or a copy by how many pixels each byte gives. Once all the data is worked out, it fills in the shared distance bits of each group of four short copies.

PCX

The header is 128 bytes (only the fields we use):

Offset	Size	Field
0	1	manufacturer, 0x0A
1	1	version (5)
2	1	encoding: 1 = RLE, 0 = none
3	1	bits per pixel in each plane (8)
4	8	x min, y min, x max, y max (width = x max - x min + 1)
12	4	horizontal and vertical DPI
65	1	planes: 1 = indexed, 3 = RGB, 4 = RGBA
66	2	bytes per line in each plane (at least the width, padded)
68	2	palette info

RLE: a byte with both top bits set (0xC0-0xFF) is a run of the next byte, byte & 0x3F long. Any other byte is used as it is. Each line holds the planes one after another, each "bytes per line" long, and the padding is dropped after reading. Runs that cross lines are accepted. An 8-bit image can end with 0x0C and 768 bytes of RGB palette. PcxPalette::Optional lets it be missing (SFF v1 sprites with "same palette").

The encoder writes version 5, RLE, one plane, lines padded to an even length, runs that never cross lines, and the palette if asked.

PNG

Through LodePNG (https://github.com/lvandeve/lodepng), in third_party/lodepng. Palette images of any bit depth give back their exact indices: they're read without color conversion and our code unpacks the indices, so palettes with repeated colors keep their indices. Other color types are read as RGBA. Indexed images are written as 8-bit palette PNGs without color conversion, with the PLTE long enough for the highest index. True color images are written as RGB (PNG24) or RGBA (PNG32).

SND sound files

snd/SndDocument.h. A 512-byte header, then a chain of sounds:

Offset	Size	Field
0	12	"ElecbyteSnd\0"
12	4	version bytes (seen: 00 01 00 01, 01 00 01 00, 04 00 00 00)
16	4	number of sounds
20	4	offset of the first sound (512)

Offset	Size	Field
24	488	unused (a comment in 87 of 96 files)

Each sound has a 16-byte header and then its data:

Offset	Size	Field
0	4	offset of the next sound (the file size for the last one)
4	4	length of the data
8	4	group (signed)
12	4	sound number (signed)

The data is a whole WAV file, kept exactly as it is. In 206 sounds the RIFF size inside the WAV doesn't match the stored length (both bigger and smaller), which doesn't matter because nothing in the SND relies on it. The sounds are stored one after another in order. Reading and writing gives back every test file byte for byte.

WAV

wav/WavFile.h. RIFF WAVE with chunks (id, 4-byte size, data, and a pad byte after odd sizes). The "fmt " chunk holds the format tag, channels, sample rate, byte rate, block align, bits per sample and maybe an extension (kept as bytes; for WAVE_FORMAT_EXTENSIBLE its sub-format is used). Samples can be unsigned 8-bit, signed 16, 24 or 32-bit PCM, or 32 or 64-bit float. All other chunks (LIST, "cue ", smpl, fact, ID3 tags and chunks from all sorts of editors, in dozens of different orders in our test files) are kept in order and written back as they were.

By default the reader puts up with the damage that's common in game files: a wrong RIFF size, a last chunk cut short after the samples, junk after the samples, a missing last pad byte, or samples cut short (kept as far as they go). WavParseOptions::strict refuses all of that.

FNT bitmap fonts (version 1)

fnt/FntDocument.h. The header is 32 bytes:

Offset	Size	Field
0	12	"ElecbyteFnt\0"
12	4	version bytes (00 01 00 00)
16	4	offset of the PCX image
20	4	size of the PCX image
24	4	offset of the text
28	4	size of the text

The image (all the letters side by side) and the text are kept exactly as they are. 17 files have 8 extra bytes between the header and the image (offset 72), and those are kept too. The text has a [Def] section (Size = w,h, Spacing = x,y, Colors = n, Offset = x,y, Type = Fixed or Variable) and a [Map] section with one line for each letter: the letter itself or a 0xNN code, then "x width" for variable width

fonts or a letter index (which can be left out) for fixed width fonts. parseFntText() reads both. A ; at the start of a map line is the ; letter and doesn't start a comment.

FNT version 2 fonts are .def text files that point to an SFF, so they aren't part of this format.

Palette files

palette/PaletteFormats.h.

- **ACT** (Adobe Color Table): 256 RGB triplets (768 bytes), maybe followed by a big-endian color count and transparent index (772 bytes). M.U.G.E.N character palettes store the colors backwards: the first triplet is palette index 255 and the last one is index 0. We checked this with sprmake2, whose default (pal.reverseact = 0) puts the last triplet at index 0, and with Kung Fu Man, whose SFF palette 1,1 is kfm.act backwards. ActOrder::Reversed reads and writes that order, and ActOrder::Natural the normal one.
- **RIFF PAL**: RIFF, the size and "PAL ", then a data chunk with a version (0x0300), a color count and four bytes per color (R, G, B, flags). Other chunks are skipped.
- **JASC-PAL** (Paint Shop Pro): the text lines JASC-PAL and 0100, the color count, then R G B for each color. CRLF or LF.
- **ACO** (Photoshop swatches), only read: big-endian. A version 1 part (version, count, then for each color a color space and four 16-bit values), maybe followed by a version 2 part that repeats the colors with UTF-16 names. RGB, HSB, CMYK, Lab and grayscale are converted.
- Palettes can also be taken from PCX and palette PNG images.

Checks against Elecbyte's tools

The oracle tests (Windows only, with OFS_CORPUS_MUGEN set to a M.U.G.E.N 1.0 folder) run:

- sff2png <file> <prefix>: saves every sprite of an SFF v1 or v2.0 file as a PNG, plus a SprMaker2 .def with the group, number, file and axis of each (the output folder has to exist). It can't open SFF 2.01 files. We write v1 and v2.0 files with every format that fits (raw, RLE8, RLE5, LZ5, PCX), linked sprites and palettes and odd sizes, and compare sff2png's pixels and palettes with ours. They all match.
- sprmake2 <def>: makes an SFF from the PNG or PCX images listed in a .def. We save our decoded sprites as PNGs, write the .def (with sprite.usepal, and sprite.removecolors = 32,255 for 5-bit sprites, which SprMaker2 only makes from a 32-color [Pal] entry), and read the SFF it makes. RLE8 and LZ5 sprites (all 279 indexed Kung Fu Man sprites too) come back pixel for pixel. So do RLE5 sprites, except the ones hit by the RLE5 bug above, and those read exactly the way sff2png reads them. Our writer also gives back the SFF files sprmake2 makes byte for byte.

What we learned from the tools: sff2png ignores a link's palette when its size doesn't fit (see Links above), raw sprites in the translated block make it refuse the file, and SprMaker2 2.00 writes 0 as the fourth palette byte and turns ACT files around by default.